

End-to-end approach to detect food on a compartmentalized plate

Louis VANDECRUYS

Arno VAN EETVELDE

Supervisor(s): Bart Vanrumste, Hans
Hallez

Co-supervisor(s): Sunilkumar Telagam Setti

Master Thesis submitted to obtain the degree
of Master of Science in Engineering
Technology: Electronics Engineering

Academic Year 2021 - 2022

End-to-end approach to detect food on a compartmentalized plate

Van Eetvelde Arno, Vandecruys Louis

Master in Electronics Engineering, Faculty of Engineering Technology, Campus GROUP T Leuven, Andreas Vesaliusstraat 13, 3000 Leuven, Belgium

Supervisor(s): Bart Vanrumste, Hans Hallez

Electronics Engineering, Faculty of Engineering Technology, Campus GROUP T Leuven, Andreas Vesaliusstraat 13, 3000 Leuven, Belgium, <bart.vanrumste@kuleuven.be, hanz.hallez@kuleuven.be>

Co-supervisor(s): Sunilkumar Telagam Setti

, <sunilkumar.telagamsetti@kuleuven.be>

ABSTRACT

Today, many people suffer from complications that are related to food intake. Obesity can lead to different kinds of heart disease, which is still the number one cause of death worldwide. But overeating is not the only food-related problem. Many people, such as elderly and hospitalized patients, suffer from malnutrition which results in a shortage of nutrients and calories. In the last decade, several machine learning methods have been proposed for automated approaches regarding nutrition measurement. In this thesis, we use machine learning algorithms to recognize the food items placed on a compartmentalized plate. We believe this can help people with their dietary assessment in the future. We employ different state-of-the-art deep-learning techniques, namely convolutional neural networks and vision transformers, and propose an automated approach for recognizing the food items on the plate. The different steps of our method are as follows. The food is placed on a compartmentalized plate, after which a picture is taken with a smartphone and afterwards fed into the deep-learning models. The models segment the food items placed in the compartments and give us a prediction for each type of food present on the plate. In this thesis, we conclude that convolutional neural networks and vision transformers can be combined in an ensemble for better overall performance and that this performance translates well to real-life custom data acquired using smartphones in different environments.

1 INTRODUCTION

Food intake can have a major impact on a person's health. Excessive calorie consumption leads to obesity (Church et al., 2011), which the World Health Organization (WHO) defines as "abnormal or excessive fat accumulation that may impair health". Obesity has been found to be the cause of numerous cardiovascular diseases, as it contributes directly to cardiovascular risk factors such as hypertension and type two diabetes (Powell-Wiley et al., 2021). One study conducted on female nurses measured an increased risk of developing coronary heart disease of up to three times when gaining excessive weight, compared to healthy weight ranges (Willett et al., 1995). Since 1975, obesity has nearly tripled worldwide (WHO, 2021). With heart-related diseases still being the number one cause of death worldwide (WHO, 2020), efforts have been made to aid people in controlling food intake to battle the obesity epidemic. The inverse is also true: many people suffer from malnutrition which can be detrimental to one's overall health (Müller and Krawinkel, 2005; Fine-stone et al., 1995; Evans, 2005). Machine learning can help make automated approaches to combat both these issues. Computer vision in particular has been used to aid other fields in solving real-world problems (Sarker, 2021). In the past decade, many studies and research papers have emerged that use data processing techniques on food-related data (Chaves et al., 2021a), showing that there is an increased interest in applying Artificial Intelligence algorithms to nutrition. Examples of using machine learning in this context are food type detection (Kagaya et al., 2014), eating activity recognition (Rashid et al., 2021), food intake assessment (Chaves et al., 2021b), etc. Automatic food type detection and intake assessment can facilitate the process of counting the consumption of calories and macro-nutrients, which is conventionally done by hand. Performing the counting manually is a tedious process that may cause people to refrain from using it as a technique to reduce calorie intake. This is where machine learning can be used to aid the process and motivate more people to live a healthier lifestyle.

In this thesis, a machine learning approach is taken to ease the first steps of measuring food intake. The pipeline can be summarized as follows: a plate that contains three compartments is used and three different types of food are placed onto the plate. A top-down picture of this plate is taken using any type of smartphone (image acquisition). The food present in the three compartments is then automatically segmented (segmentation). Afterwards, a model classifies the three different foods that are present on the plate (classification). Using this procedure, we hope to aid research in automating the process of nutrition measurement. To our knowledge, we are the first work to use

a compartmentalized plate, which allows for more accurate food segmentation. We also create a small in-house dataset and combine different classification methods for better results.

2 LITERATURE STUDY

This chapter provides a brief overview of existing literature for both classification and segmentation.

2.1 Related Classification Works

Classification is the task of determining which class a set of data points belongs to. In this thesis, the set of data points are pixels of a food image, and the class is a type of food. Several types of machine learning approaches have been used to tackle automatic food classification. These approaches can be divided into two categories: traditional machine learning and deep learning.

2.1.1 Traditional Machine Learning

Traditional machine learning algorithms rely on hand-crafted features that are created by experts and on which the chosen algorithm can train and afterwards perform its prediction. Examples of traditional machine learning are k-Nearest Neighbour (k-NN), Support Vector Machines (SVM), Decision Trees (DT) and others. The self-proclaimed "first food image classification which can be applied for practical use" relied on such traditional approaches (Joutou and Yanai, 2009). This paper makes use of Multiple Kernel Learning (MKL) which is an extension of SVM that allows for training with multiple kernels. Using multiple kernels allows for feature fusion, allowing the SVM to detect multiple features at once, such as colours, textures and Bag of Features (BoF). Using this method they were able to achieve a classification accuracy of 61.5% on their own custom dataset, consisting of traditional and non-traditional Japanese foods. Another paper using SVM released the now commonly used and publicly available dataset "Food-101" (Bossard et al., 2014). Together with this dataset, the authors released a novel way of approaching image classification of food images. Using a Random Forest (RF) algorithm with linear classifiers, they were able to mine discriminative components of images. Discriminative components are a set of patterns or pixels that distinguish a certain class from other classes in the dataset. Mining for discriminative components was not a new technique, but by utilising Random Forest, the authors were able to mine for all classes at once, meaning the nodes of the forest can share information among them. Using the leaves (final nodes containing class sam-

ples) of the forest, they could now train component models for each class. To classify the images, only the superpixels (group of pixels that share common characteristics) of this image had to be scored against all component models derived from mining. After the scoring, a multi-class SVM was used to do the final classification. Using this technique, the authors were able to outperform various other state-of-the-art component-mining approaches applied to their newly constructed dataset, reaching an accuracy of 50.76%. However, their technique was outperformed using a convolutional neural network (CNN), which is a deep learning technique that achieves tremendous results in the computer vision field. This resulted in an accuracy of 56.40%.

2.1.2 Deep Learning

Deep learning, compared to traditional machine learning, doesn't rely on hand-crafted features to be able to learn from images. A deep learning model can teach itself to derive features from a picture, only needing the pixel values as input (for further elaboration, see chapter 4). Since CNN's are so powerful when applied to image processing, they have become the de facto standard for achieving state-of-the-art results in food classification, only recently being challenged by vision transformers.

2.1.2.1 Convolutional Neural Network (CNN)

The existing works using CNN in the context of nutrition started appearing from 2014 onwards. A paper by Kagaya et al. (2014) constructed a dataset of the ten most common foods on a food-logging app and applied the CNN deep learning technique to it. After comparing the CNN to an SVM with handcrafted features (such as color spatial pyramid matching), results once again showed the deep learning method superiority over traditional methods, with an average accuracy of 73.70% compared to 60.47%. In 2016, Liu et al. applied a CNN-based algorithm to the aforementioned Food-101 dataset (Liu et al., 2016). Their CNN architecture was based on the GoogLeNet model, which is a model built upon concatenated Inception modules. The authors optimized this model by adding extra layers containing 1x1 convolutions. These 1x1 convolutions allow dimension reduction between different layers while at the same time increasing model depth. This results in fewer computational bottlenecks while allowing for better visual feature detection due to depth. This was proven by the results their model achieved on the Food-101 dataset. Reaching an accuracy of 77.4%, they outperformed Bossard et al. by 21%. Martinel et al. proposed another approach for food recognition and achieved an ac-

curacy of 90.27% on Food-101 (Martinel et al., 2016). To accomplish this, the authors made use of residual learning, which is a technique that solves issues when training very deep neural networks, developed by Microsoft (He et al., 2015) (the model architecture derived from this technique was coined ResNet and has since become very popular in image recognition). In combination with this, the paper by Martinel et al. used a special type of convolution, called a slice convolution. This type of convolution is specifically designed to detect horizontal features, which are abundantly present in pictures of food according to the authors.

With the evolution of computational power and the possibility to train on huge datasets, scaling up deep neural networks for better results has become easier and easier. This can be proven by looking at the image classification leaderboard on ImageNet. ImageNet is a dataset containing 14,197,122 images that have been annotated by hand, on which image classification techniques can be trained and benchmarked (Deng et al., 2009). However, when we refer to a (classification) model that is pre-trained on ImageNet, we usually mean ImageNet1k, which is the one thousand class variant of ImageNet with around 1.2 million images. This dataset has become the standard for evaluating the image classification performances of newly developed neural network architectures and other image classification techniques. In 2014, a research team at Google released a paper in which they describe GoogleNet, a convolutional neural network containing 22 layers and 6.798 million trainable parameters, which achieved a top-5 accuracy of 93.33% on ImageNet (Szegedy et al., 2014). In 2015, this accuracy was outperformed by 2.1% by ResNet-152, the 152 layer variant of the aforementioned ResNet network architecture containing 60.344 million parameters (He et al., 2015). Although achieving a substantial increase in accuracy, this came at a cost of almost ten times as many parameters. Unsurprisingly, the current ImageNet state-of-the-art CNN model followed the trend of increasing parameter count in favour of higher accuracy. The model called NFNNet-F4+ not only has 527 million parameters but is also pre-trained on an internal Google dataset called JFT-300M, which is not publicly available, consisting of 300 million images (Brock et al., 2021). Due to these depth and parameter developments in CNN's, many models can now generalize well on new datasets without needing to be designed from scratch specifically for them. This is shown in the current state-of-the-art on food classification, which was achieved by Google's EfficientNet in combination with a novel optimizer (Foret et al., 2020). With around 480 million parameters pre-trained on ImageNet and additional images such as the aforementioned JFT-300M dataset, their top-1 accuracy of 96.18% on Food-101 is the current state-of-

the-art to the best of our knowledge.

2.1.2.2 Vision Transformer (ViT)

Vision transformers are computer vision models based on the transformer architecture widely used in natural language processing (NLP) tasks (Dosovitskiy et al., 2020). Since its release in 2020, ViT models have gained much popularity as they are the first in being able to challenge the performance of CNN's in computer vision related tasks. At the time of writing, the number one spot on the ImageNet leaderboard is being held by ViT-G/14, a large-scale vision transformer with 1,843 million parameters (Zhai et al., 2021). This model produces a 90.94% top-1 accuracy on ImageNet, achieved by fine-tuning ViT-G/14 models on the dataset using different hyperparameters (learning rate, augmentations, weight decay,..) and afterwards averaging weights of these models into one model instead of discarding all underperforming models. The resulting state-of-the-art models were coined "Model Soups" (Wortsman et al., 2022). One example of vision transformers being used in nutrition context is (Nijhawan et al., 2022). In this paper, the authors propose a hybrid ViT/SVM model to classify Indian foods in a custom dataset. The hybrid model is an ensemble of a ViT feature extraction with an SVM classifier and an SVM classifier with hand-crafted features. The handcrafted features were histogram of gradients (HoG), local binary patterns (LBP) and Gist encodings. The authors compare the hybrid ViT/SVM ensemble to other ensembles, such as CNN/SVM and CNN/RF. Results showed that the hybrid ViT/SVM model outperformed all other models, suggesting that features extracted from vision transformers can be better than CNN-extracted features in certain contexts.

2.2 Related Segmentation Works

Image segmentation can be defined as dividing images into subgroups called image segments. An image segmentation model will group pixels of an image that it thinks belong together. Image segmentation can be used for locating objects in an image, compressing an image, detecting boundaries in an image, etc. The main goal is to make the image simpler for other following image processing tasks, such as image classification for example. Both traditional and deep learning techniques can be used to perform image segmentation, this thesis only focuses on the deep learning approaches. Long et al. were the first to use fully convolutional networks (FCN) for image segmentation (Long et al., 2014). An FCN only employs layers that are connected locally like max-pooling and convolutions, making the network able to work for variable input

dimensions, which is not possible for conventional CNN's. FCNs also use an encoder with upsampling architecture to make semantic image segmentation possible. In Chen et al. (2014), the authors propose deep convolutional neural nets (DCNN) in combination with fully connected conditional random fields (CRF) to achieve semantic image segmentation. They employ the "atrous" algorithm also known as dilated convolutions in the last four layers of the model to achieve dense CNN feature maps. This allows them to return to the original image size at the testing stages of the model by performing bilinear interpolation, which requires little computational power. This upscales their image eight times while avoiding sparse output resolution. In contrast to the previous approach by Long et al. where an extra upsampling architecture had to be added to reach the required image resolution, their architecture was simpler and thus easier to train (several days of training compared to ten hours of training). Not only is their model easier to train, it also achieved better Intersection over Union (IoU) results on the PASCAL VOC 2012 test set, which is a popular dataset for benchmarking image segmentation methods (Everingham et al., 2010). Chen et al. coined their model "DeepLab" and later released updated versions in which they made several improvements to their original model, called DeepLabv2, DeepLabv3, DeepLabv3+ (Chen et al., 2018). These papers focus on semantic image segmentation, meaning pixels that belong to the same class are treated as one entity (e.g. multiple apples will form one entity and will just be seen as apples by the model). This thesis requires instance segmentation, meaning differentiating distinct instances of objects that belong to the same class (e.g. classifying the group of apples as apple 1, apple 2, etc.). In He et al. (2020), the authors combine two image segmentation techniques, Faster Region Based CNN (R-CNN) and FCN to get Masked R-CNN, a novel technique that provided state-of-the-art results on another benchmark dataset called COCO (Lin et al., 2014).

2.2.1 Food segmentation

In the context of nutrition, a paper by Wu et al. developed a novel dataset for benchmarking image segmentation of food items (Wu et al., 2021). In this paper, they compare three different segmentation modules. Firstly, dilation-based segmentation (as described before in (Chen et al., 2014)). Secondly, Feature Pyramid Network (FPN) based, meaning a pyramid of stacked feature maps is formed (feature maps from shallow and deep layers). These layers are merged and followed by a mask predictor. Lastly, transformer-based segmentation, meaning the attention layers of the transformer architecture are used to aid in segmentation. These three methods were used in com-

combination with a Recipe Learning Module (ReLeM), a module that encodes information of food recipes into the vision encoders, which is used to initialize the segmentation encoder weights of the three aforementioned methods. The authors call this method "multi-modality knowledge transfer", meaning textual and visual knowledge is shared due to inserting the recipe information into the visual feature space of the image. By combining the segmentation module and the Recipe Learning module, the authors were able to achieve sophisticated segmentation results on their custom dataset, also showing that their ReLeM with segmenter architecture increased performances compared to a standalone segmenter module. Note however that this paper covers semantic segmentation, meaning there is no differentiating between instances of the same ingredient in a picture (which makes sense, as this is an extremely difficult task in the context of recipes).

3 OUTLINE OF PROPOSED APPROACH

As mentioned before, our approach consists of three main steps: image acquisition (i), image (instance) segmentation (ii) and image classification (iii). An example of the pipeline can be seen in figure 8.1.

3.1 Image acquisition

The initial idea of this thesis was to classify and segment food images containing meals exclusively from the Alma student restaurant, located at Tienestraat 115, 3000 Leuven in Belgium. However, meals from other restaurants as well as home-cooked meals were also used later on. The meals had to be divided into three main components of the recipe. An example is shown in figure 3.1. On the left-hand side, the empty plate is shown. The middle image consists of a meal divided into three parts: a chicken leg, cooked vegetables and roasted potatoes. These pictures were not made in a professional lab environment, as there are some shadows still present in the images, as well as the angle of the picture taken not being perfectly perpendicular to the ground. In the right-most image, some meal components can be seen overflowing into the other compartments. These conditions make the tasks slightly more difficult. Figure 3.2 denotes the usage of these images with respect to the models.

3.2 Image segmentation

After the image is taken, it is segmented. The goal is to be able to differentiate the 3 instances of food on the plate and extract them from the picture. An example of this process can be seen in figure 3.3. To train the segmentation

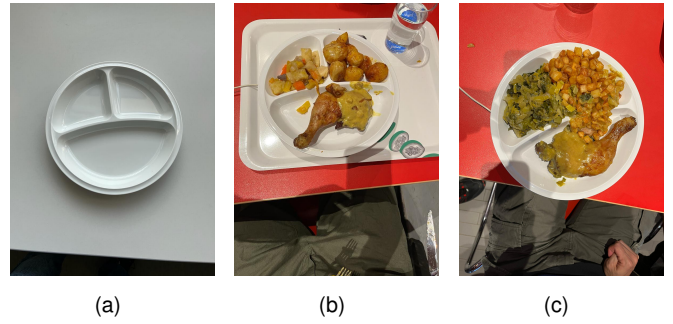


Figure 3.1: Alma food examples with (a) compartmentalized plate, (b) chicken leg with cooked vegetables and roasted potatoes and (c) chicken leg with leek and potato cubes

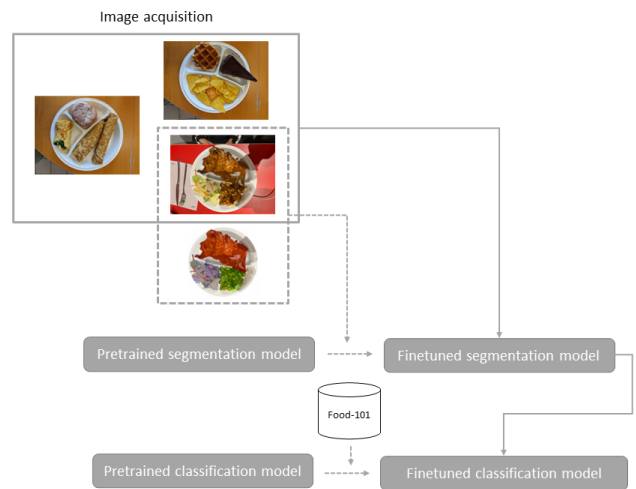


Figure 3.2: Representation of image acquisition in our approach. A dotted line denotes training and a full line denotes inference.

model, we created an in-house dataset of meals divided into 3 parts with their corresponding masks. By doing so, the model gets good segmentation results using only a small in-house dataset. To perform this segmentation, a Mask R-CNN is used. Details of this architecture are described in chapter 5.

3.3 Image classification

After the segmentation, a model takes each instance and classifies them separately. For now, the models are only able to classify 101 different types of food, meaning some food types are not able to be recognized. The dataset that was used to learn the classification is Food-101 (Bossard et al., 2014). This dataset is further elaborated upon in chapter 7. An example of the classification process is shown in figure 3.4. This thesis explores multiple architectures to recognize the food categories, such as Vision Transformer (Dosovitskiy et al., 2020), EfficientNet (Tan

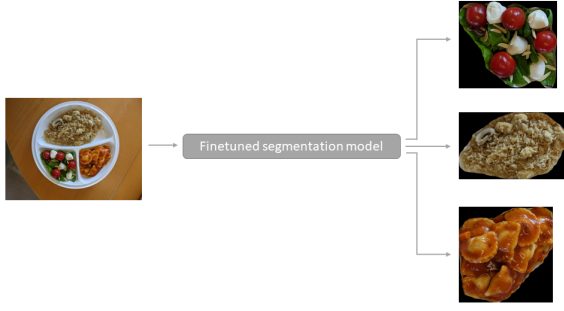


Figure 3.3: Simplified representation of segmentation

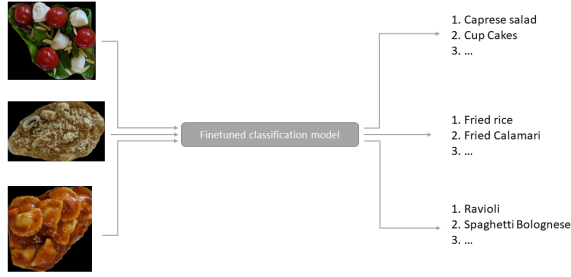


Figure 3.4: Simplified representation of classification

and Le, 2019) and Swin Transformer (which is a type of vision transformer) (Liu et al., 2021). Chapters 4 and 6 explain the details of the aforementioned networks.

4 BACKGROUND

This chapter discusses the two core machine learning techniques behind the used models.

4.1 Convolutional neural networks: a brief explanation

A convolutional neural network is a deep learning technique that takes an image as an input and can directly learn features from this input without the need for external feature crafting. There are three main layers that are core to the CNN architecture: a convolution, a pooling and an activation layer. When the modeling problem is classification, which is usually the case, a fully connected layer (or multiple) is also employed as the last layer of the model, also called a dense layer, followed by a last activation layer.

Initially, a CNN takes an image as an input. This image is represented as a 3D array of pixel intensities, with height and width corresponding to the image size and the depth being the number of channels. The number of channels in this case is three, one for each of the red, green and blue (RGB) pixel intensities. Each value of the input array

ranges from 0 to 255, 255 being the most intense value of a colour.

The convolution layer applies a kernel, also called a filter to this input. A kernel is a smaller array of values, also called weights, which are initialized arbitrarily. These weights are adapted by the model through backpropagation. The kernel is applied by performing a dot product between a local part of the input array and the values of the kernel. This requires the kernel to be three-dimensional, to match the dimensions of the local part of the input image. This dot product results in one value of a feature map. The kernel then shifts by a certain number of pixels over the image. This amount is called the stride (usually chosen as 1). After the shifting happens, another dot product is done and a second value is added to the feature map. This is done until every pixel of the input array is covered and the feature map is filled with values. This feature map now represents a feature. When the CNN wants to detect more features in the same layer, it uses multiple kernels. The result is now another 3D array, with depth equal to the number of kernels that were used to perform the dot product. Note that a convolution will always decrease the height and width of the input array (except for 1x1 convolutions). To keep height and width the same, the input array can be padded on the outside with extra values (usually 0). The computation of the convolution is shown in Eq. 4.1 (Chang and Jin, 2017).

$$x_{i,j,k}^{(l)} = \sum_a \sum_b \sum_c w_{i,j,k}^{(l-1,f)} y_{i+a,j+b,k+c}^{(l-1)} + bias^{(f)} \quad (4.1)$$

Where $y_{i,j,k}^{(l-1)}$ are the output values of the previous layer. With l being the current layer, and i, j, k being the coordinates of the node. $w_{i,j,k}^{(l-1,f)}$ are the weights of the kernel that is applied to the outputs of the previous layer. a, b and c represent the coordinates of the weights of the kernel. $Bias^f$ is a bias that is added to each value of the output array. Each output channel has its own bias that is learnable by the network through backpropagation. The output of the layer is then given by Eq. 4.2.

$$y_{i,j,k}^{(l)} = \sigma(x_{i,j,k}^{(l)}) \quad (4.2)$$

Where σ denotes the non-linear activation function, and $y_{i,j,k}^{(l)}$ is the output of the current layer. The non-linear activation function is almost always a Rectifier Linear Unit or ReLu in convolutional neural networks. This function turns all negative values to zero and keeps positive values the same. This reduces computation complexity and gets rid of the vanishing gradient problem that other activation functions such as the sigmoid suffer from. Furthermore, if the dimensions of the input array can be represented as $[w, h, n_c]$, with w being width, h being

height and n_c being the number of channels, the dimensions of the resulting feature array are represented as $[\frac{n+2p-f}{s} + 1, \frac{n+2p-f}{s} + 1, n_f]$ where n is the size of the input array, p is the amount of padding, f is the kernel size, s is the stride and n_f is the number of kernels used.

After a convolution layer, usually a pooling layer follows. This is a downsampling layer used to decrease the dimensionality of the feature maps. Aside from decreasing the number of learnable parameters, the pooling layer allows the network to be invariant to translations in size, scale and shape. The two most popular variants of pooling are max-pooling and average-pooling, but for CNN's, max-pooling is almost always the variant of choice. Max-pooling takes a small sub-sample of the input array, say a 2x2 square, then takes the largest value of this sample and puts it in the output array. This means four values at the input have been reduced to one. Just like a kernel, the stride and size of the max-pooling can be set as a parameter.

A convolutional neural network consists of many convolutional and pooling layers in a row, with earlier layers detecting low-level features such as edges and circles, while later layers can detect high-level features such as a nose or eyes in a human face. At the end of the network, the final feature array is flattened, meaning all values are unrolled into a 1D array. This array is connected with 1 or more fully connected layers. In multi-class classification, the final fully connected layer should have the same number of output nodes as the number of classes that are being detected. These values are then passed through a last activation layer, a softmax layer, which squashes the 1D array to a 1D array containing only values ranging from 0 to 1, where all values add up to 1. The output values can now be seen as the probability for each of the target classes resulting from the input image. A graphical representation of a small CNN can be seen in figure 4.1.

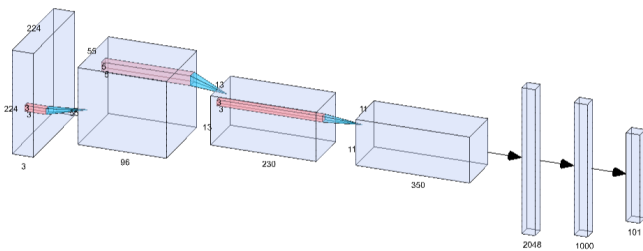


Figure 4.1: CNN-example

4.2 Vision transformer: a brief explanation

When the transformer architecture was introduced in 2017, it took the natural language processing field by storm. Now in 2022, the vision transformer and derivatives thereof are rivaling CNN's in multiple computer vision

tasks (Wortsman et al., 2022). The paper that introduced the vision transformer architecture is titled *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale* (Dosovitskiy et al., 2020). This title refers to the original transformer architecture requiring a sequence of words as the input. The authors used the same approach but applied it to an image: first, they divide the image into equally sized patches and afterwards put them in a sequence, just like a sequence of words (a sentence) would be used in the original transformer architecture (Vaswani et al., 2017). In the ViT paper, they divide their images into 16x16 patches and 14x14 patches and compare their performance. If the dimensions of the input image are $[w, h]$ then the number of patches can be denoted as $N = \frac{hw}{p^2}$ where p is the resolution of each patch. 16 by 16 or 256 patches and an image resolution of 224 (like in the paper) would result in a patch resolution of 14. These patches are then flattened into a sequence of 1D vectors, so a patch with resolution 14 would result in a vector with dimensions $[1, 588]$ ($14 \cdot 14 \cdot 3 = 588$) with N patches creating an input matrix with $[N, 588]$ dimensions. A trainable linear projection is then used to map the sequence of 1D vectors into a sequence of latent vectors of size D , with weights of the linear projection being shared among the input space. This sequence of vectors now represents the patch embedding. Another vector is prepended to this sequence, which serves as a learnable class embedding. Only the output of this embedding will be used to perform the classification at the end of the model. This creates an $N + 1$ by D input matrix, to which a matrix of learnable 1D positional embeddings is added. This is done because the transformer architecture has no inherent positional sense of the patches, so extra vectors with learnable values are added to the input so the model can teach itself the positions of the patches.

After the positional embeddings are added, the matrix is fed into the transformer encoder, which consists of multiple layers of a Multi-Head Attention and Multilayer Perceptron (MLP) block. Note that the shape of the input matrix does not change throughout the whole transformer architecture. A layer starts by applying a layer norm to normalize the distribution of the matrix values as they pass through each layer, resulting in smoother gradients, faster training and better accuracy (Xu et al., 2019). Then the Multi-Head Attention follows. Here, the vector dimensions are first enlarged by a learnable linear projection. The resulting matrix is called the qkv matrix and is afterwards divided into three equal parts: the query, key and value matrices. In our example, these three matrices are of shape $[257, \frac{D}{3}]$, and are further reshaped into dimensions $[12, 257, \frac{D}{36}]$ to create the 12 attention heads. In theory, each of these attention heads would learn different information, thus giving the encoder more representation power. The k matrix rep-

representing the keys is transposed into a $[12, \frac{D}{36}, 257]$ matrix to get the correct dimension for the upcoming dot product. Afterwards follows the attention operation, a scaled-dot product, as per (Vaswani et al., 2017) in Eq. 4.3.

$$Attention(q, k, v) = softmax(\frac{qk^T}{\sqrt{d_k}})v \quad (4.3)$$

The scaling factor in $\sqrt{d_k}$ was added to reduce the outcome values of the dot-product, which would result in a very low gradient due to the usage of a softmax, also known as the vanishing gradient problem. Furthermore, the output of the attention operation is transposed and flattened to get a three-dimensional matrix with identical dimensions to the input, namely $[257, D]$, where 257 is the number of patches + 1 and D was formed by flattening and projecting the 3D patch. This is then passed through a last fully connected layer, with the same input and output dimensions, to process the information gained by the attention heads. This results in an output matrix containing vectors with information on how each patch in the image attends to all other patches from the image. This matrix is added to the original input matrix, which is called a residual or skip connection. The output of this residual connection passes through a layer norm and afterwards into the MLP block, consisting of a 2-layer fully-connected feed-forward network with a GELU activation in between. The output of this MLP block again is added to the input of the MLP block with a residual connection. The vector at the first index, the class token that was prepended before the positional embedding, can now be used for classification by attaching a linear layer with output dimension equal to the number of classes.

The attention module makes it so the vision transformer attends to global information in earlier layers, while this is impossible for CNN's as they are heavily localized. The transformer architecture was shown to attend to image regions that were important to classification, as can be seen in figure 4.2.

This concludes every operation that is done by a Multi-Head Attention block followed by an MLP block. Since the output and input dimensions are the same, layers of this sequence of two blocks can be stacked after each other. Stacking these layers will cause the attention heads to attend to more global information, resulting in a better classification accuracy. The original paper performed testing to show that results increased for up to 60 layers, but started diminishing heavily at 16 layers (Dosovitskiy et al., 2020). Moreover, the authors note that vision transformers have much less inductive bias than CNN's, meaning they are not as translation equivariant and thus need more training data to challenge a CNN's performance.

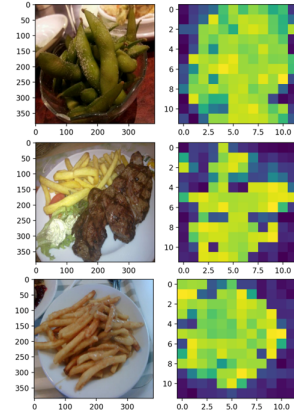


Figure 4.2: Input images with attention mechanism, made by computing the average of all attention maps at the output layer of the vision transformer

5 SEGMENTATION

To classify the food items present on the compartmentalized plate, each of the three items first have to be extracted out of the picture. Initially, it was thought this could be done with a simple circle and line detecting algorithm, since the plate is distinctly divided between the 3 compartments. However, some images that were taken had food from one compartment pouring over into other compartments, making the simple algorithm obsolete. Consequently, this thesis now uses deep learning to segment the food items. This is done with instance segmentation which combines both the task of individually detecting objects and predicting which pixels belong to the instance of each object. This thesis follows (He et al., 2020) and uses the Mask R-CNN method with a ResNet-50 Feature Pyramid Network (FPN) as the feature extraction backbone to perform the segmentation. The first stage of this network consists of a Region Proposal Network, which is a fully convolutional network (FCN) that searches for possible bounding boxes of objects in feature maps of an image. These feature maps were generated by the ResNet-50 FPN. In the second stage, these proposed regions are fed into a Fully Connected Network together with the original feature map to perform classification and bounding box regression, meaning this branch has two output vectors: one with classification scores and one with bounding box offsets. Meanwhile in parallel, an FCN branch is employed on the same feature maps and bounding boxes. This branch outputs K binary matrices for each proposed bounding box, where a value of 1 is used when the pixel belongs to class K and a value of 0 when it doesn't. During the training, it is the job of the aforementioned classification branch to conclude which of the K mask matrices will be used for the prediction.

In our application, there are only two classes: background and food. Intersection over Union (IoU) is used to evalu-

ate the performance of the segmentation model. The IoU formula is given in Eq. 5.1.

$$IoU = \frac{Area\ of\ intersection}{Area\ of\ union} \quad (5.1)$$

Where the area of intersection is the overlapping area of the ground truth mask and the predicted mask and the area of union is the total area that the predicted mask and ground truth mask occupy.

Another popular evaluation metric for segmentation models is the F1 score or dice coefficient (Zhang et al., 2014). The formula for this metric is given in Eq. 5.2.

$$F1\ score = \frac{2 \cdot (Area\ of\ intersection)}{Total\ area} \quad (5.2)$$

Where the area of intersection is the same as the one for the IoU formula and the total area is the area that the predicted mask and ground truth mask occupy without subtracting overlap.

For training the model, a 22/6/12 data split is used from the 40 pictures of food in the in-house dataset. The model is first pre-trained on the COCO dataset which contains 328 thousand images with bounding box offsets and per-instance segmentation masks. Afterwards, the model was trained for 150 epochs on the in-house dataset with Stochastic Gradient Descent as an optimizer with a learning rate of 0.005, 0.9 momentum and 0.0005 weight decay. Furthermore, the learning rate was decayed by 0.1 for every three epochs the model was trained. A random horizontal flip was used as augmentation. During testing, the image is automatically cropped to the size of the compartmentalized plate. A contrast is also applied to the input image, this makes the edges sharper and results in better segmentation accuracy. The formula for the added contrast is shown in Eq. 5.3.

$$Output = \alpha \cdot Input + \gamma \quad (5.3)$$

$$\alpha = \frac{131 \cdot (contrast + 127)}{(127 \cdot (131 - contrast))} \quad (5.4)$$

$$\gamma = 127 \cdot (1 - \alpha) \quad (5.5)$$

In this formula, the contrast is first set to 20, and if this results in insufficient output masks after segmentation, it is further increased to 40. Occasionally, the model outputs more than three masks, where some small parts of food (e.g. sauce) are viewed as a separate instance of food. In this case, the three masks of which the model is the surest are calculated, while also removing very small masks and overlapping masks, leaving us with the three desired masks.

6 CLASSIFICATION

6.1 Transfer learning

To fine-tune the chosen models for classification, transfer learning was used. This means that the network architectures already contain weights that were learned from features of images from big datasets, usually being ImageNet (Deng et al., 2009). If these weights perform well on this dataset, the knowledge that the model contains can be transferred onto a different target dataset. Especially in computer vision, this can circumvent the need for big datasets which are often required to learn and detect features in images. When transfer learning, particular layers of the models can be frozen to control which layers are still updated through backpropagation. For example: one can choose to freeze the convolutional layers of a pre-trained CNN, which means only the fully connected layers that are used for classification are trained. This means that the classification will be performed on the features that the model detects at inference with the weights that it learned for the ImageNet dataset. One can also choose to unfreeze the convolutional layers, telling the model that it should adapt the feature detection to a different dataset, starting from the pre-trained weights. Most of the training was done on supercomputers of the Vlaams Supercomputing Centrum (VSC), a cloud computing service offered by KU Leuven. Here we had access to multiple NVIDIA P100 GPUs and occasionally two NVIDIA V100 GPUs, albeit at certain times in the day with often long waiting times.

6.2 Models

6.2.1 VGG16

Initially, some baseline models were considered to train and perform the classification. The first chosen model is VGG (Simonyan and Zisserman, 2014). Released in 2014, it was the first paper to consider scaling the depth (number of layers) of CNN's and how this scaling can improve performance on classification tasks. The 16-layer variant was chosen, consisting of thirteen convolution layers and three fully connected layers with pre-trained ImageNet weights. The last three layers were replaced with a custom classifier, consisting of one fully connected layer, a ReLu activation, a dropout layer and a final fully connected layer. Afterwards the convolutional layers are frozen to perform transfer learning.

The data split in a custom split, 80/10/10. Some augmentations were employed on the training split, such as random rotation, horizontal flipping, etc. A learning rate of 0.002 was employed with no decrease in further epochs,

Adam was the chosen optimizer. Using this method, the first model reached a top-1 accuracy of 45.13%.

6.2.2 ViT

For the second baseline model, a vision transformer is used. The ViT-B/16 is used, which is the base vision transformer variant with twelve transformer encoder layers and a patch size of 16 by 16. This model was pre-trained on ImageNet21k, the bigger variant of the ImageNet1k dataset. For this model, the last MLP layer that performs the classification on the class token is replaced with the same custom classifier as the VGG. Using the same training methods that were employed on the VGG16 model, a top-1 accuracy of 75.02% was reached.

6.2.3 EfficientNet

EfficientNet is a family of CNN models that take a different approach to scaling up architectures (Tan and Le, 2019). Before EfficientNet, models were generally scaled-up by increasing width (number of channels), depth (number of layers) or resolution (size of input image). Instead of doing this arbitrarily as other research was doing, the authors looked for a compound scaling method where depth, width and resolution were scaled with a shared coefficient, while also putting a limit on computational resources such as memory usage and FLOPS (Floating-point Operations Per Second). The formulas for their new scaling method were as such:

$$d = \alpha^\phi \quad (6.1)$$

$$w = \beta^\phi \quad (6.2)$$

$$r = \gamma^\phi \quad (6.3)$$

Where d , w and r are depth, width and resolution, respectively. α , β and γ are constants that can be found by performing a grid search. ϕ can be chosen by the user and is only limited by the amount of resources that are available. In the paper, the number in the model name (e.g. EfficientNet-B5) represents ϕ . Besides these values, a sophisticated baseline architecture is essential to increase performance when scaling. A scaled-up AlexNet (Krizhevsky et al., 2012) is highly unlikely to outperform a scaled-up ResNet, since ResNet contains newer and better layer operations. Their baseline architecture can be seen in figure 6.1.

This baseline architecture was found by performing a neural architecture search that optimizes both accuracy and FLOPS. Remarkably, the baseline structure is rather sim-

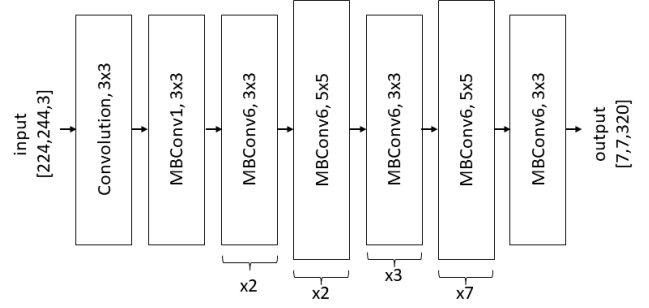


Figure 6.1: EfficientNet-B0

ple. The most notable operation is the usage of MB-Conv block, also known as inverted residual block. Essentially, this block widens the input dimension and narrows it down again while also employing a skip connection. It was first introduced in the release of MobileNetV2, a paper that searches for lightweight solutions in convolutional neural networks, making them more efficient for usage in mobile applications and embedded devices that have limited computing power (Sandler et al., 2018). This is partially done by using depthwise convolutions which decrease the number of parameters and latency in models. Depthwise convolutions divide the filters and inputs in the channel dimension and perform the convolutions channel-wise instead of over the whole width dimension (Chollet, 2016). As shown in (Kaiser et al., 2017), a normal convolution with kernel size k and channel width c would result in kc^2 parameters while a depthwise separable convolution would result in only $kc + c^2$ parameters, which is a substantial difference, seeing as EfficientNet-B5 contains layers with more than 3,000 channels. Furthermore, 1x1 convolutions are employed to increase the channel width. Lastly, squeeze and excitation optimization (SE) is used, which is essentially a global average pooling followed by a fully connected network and a scaling layer that results in an increased understanding of interchannel relationships (channel attention). Combining these operations, (Tan and Le, 2019) reached state-of-the-art performances on ImageNet and other transfer learning datasets, including Food-101, while drastically decreasing the number of parameters and FLOPS.

The EfficientNet-B5 variant was chosen to be fine-tuned on the Food-101 dataset. From the experiments, it was found that bigger variants require too much training time for proper fine-tuning. The creators of the dataset provide a 75/25 data split for training and testing. 20% of the training set was split randomly and used as a validation set. Numerous augmentations were used such as brightness, flipping, zooming, warping, etc. Examples of these augmentations are shown in figure 6.2. The chosen optimizer was Adam with Cross-entropy loss as the loss function.



Figure 6.2: Examples of augmentations on an input image

For this model, rather than arbitrarily choosing the learning rate, a different approach was taken. Since the training set is quite big ($\sim 4\text{GB}$), brute-forcing the learning rate and repeatedly training from scratch was not an option, as waiting queues for GPUs on the VSC computers could last for multiple hours at times. A small learning rate would result in the model taking too much time to converge, while a large learning rate can make the model converge into a suboptimal local minimum in the loss landscape, or even diverge (Bengio, 2012). On the other hand, a large learning rate with annealing will almost always generalize better than low learning rates, despite lower learning rates optimizing the training loss faster (Li et al., 2019). To combat the tedious tuning of learning rates, Smith (2015) proposed a learning rate finder. This method makes use of Cyclical Learning Rates, and can potentially replace learning rate tuning. A Cyclical Learning Rate means increasing and reducing the learning rate linearly between lower and upper boundaries *while training*. To find a good initial learning rate, the author proposes to train the model for a small number of iterations with mini-batches, while increasing the learning rate from a low value (e.g. $10\text{e-}7$) to a very high value (e.g. 10), changing the learning rate every iteration. The losses are recorded and plotted against the learning rates, as can be seen in figure 6.3. The learning rate should be chosen slightly before the minimum, since the minimum is on a tipping point of increasing and decreasing the loss. In figure 6.3, anywhere from $2\text{e-}3$ to $4\text{e-}3$ will be adequate. This value is now used as the upper bound of the Cyclical Learning Rate while the lower bound should be between three to ten times lower, as recommended by Leslie N. Smith in a follow-up paper (Smith and Topin, 2017). For training, the author proposes the one-cycle method, meaning that the learning rate is increased from the lower bound to the upper bound in the first half of iterations and decreased again to the lower bound in the second half of iterations, leaving a small number of iterations at the end to further reduce the learning rate to very small amounts. Smith and Topin (2017) showed that models getting suggested a high learning rate from the

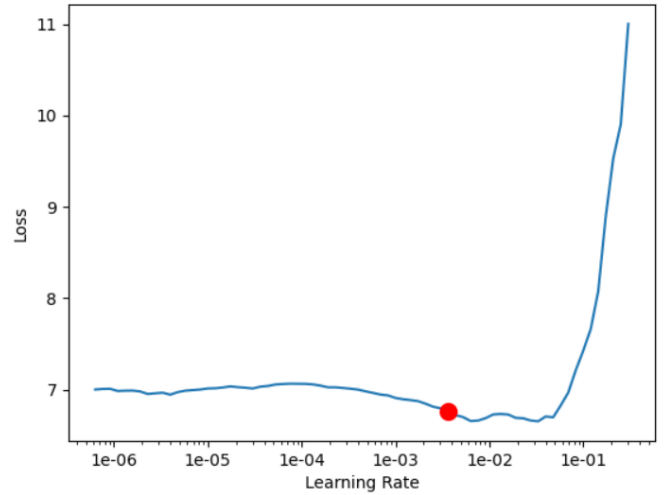


Figure 6.3: Learning rate finder on EfficientNet-B5, created using the fastai library

learning rate finder can benefit from a phenomenon they coined "super-convergence", where the models can converge to an optimal solution much quicker with a cyclical learning rate than when conventional learning rates are used. This method was subsequently used to fine-tune EfficientNet-B5, performing a learning rate finder, afterwards setting this learning rate as the upper bound and training the model using the one-cycle method. The model is trained in intervals of four epochs, performing a learning rate finder at every interval and training for a total of sixteen epochs.

6.2.4 Swin Transformer

In the original Vision Transformer, every patch attends to all patches globally, increasing required computation resources drastically with input image size. To combat this, Liu et al. (2021) propose a different attention method, where patches are further grouped into non-overlapping windows and attention per patch is only computed for patches that belong to the same window. To correct for the cross-window communication loss, Swin Transformer blocks always come in pairs, where the first block uses the regular windows while the second block shifts the windows diagonally with a step of $(\frac{M}{2}, \frac{M}{2})$ where M is the window size. Aside from partitioning the patches in windows, each stage of the Swin Transformer is initiated by a patch merging layer, where 2×2 neighbouring patches are merged, reducing the number of tokens by four, but doubling each token's length. The Swin Transformer architecture family always consists of four layers, where larger models increase the length of the linear embedding which results in more heads being computed in the Multi-Head Self Attention layer. For fine-tuning on the Food-101 dataset, the Swin-L (large variant) model was chosen, which merges

4x4 patches and uses a window of size 12 with an input image resolution of 384. For image pre-processing and training, the same methods as the EfficientNet-B5 model are used. The model converged after eight epochs of training.

6.3 Average ensemble

The two aforementioned models are combined by taking the output probabilities of the EfficientNet-B5 model and the Swin Transformer model and computing the average probability per class, taking the highest average probability as the final output. This combination is called an average ensemble and resulted in the highest accuracy for the classification task. During testing, Test Time Augmentation (TTA) is used, meaning the model predicts on ten augmented versions of the image and chooses the class with the highest average probability.

7 DATASETS

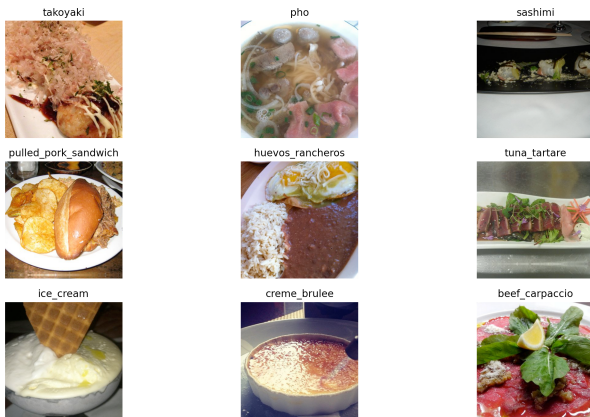


Figure 7.1: Food-101 examples

7.1 Food-101

The first dataset is used for classification of the food and was taken from (Bossard et al., 2014), which was coined Food-101. Since its release, the dataset has become very popular as it is one of the only food datasets containing a large number of labeled food images from popular categories while being a balanced dataset. The dataset has 101 classes, with a total of 101,000 images divided into 75,750 training images and 25,250 testing images. The dataset also contains some noise in the form of wrongly labeled images, as the creators of the dataset believed that machine learning models should be able to adapt to this noise if the goal is to be successful in real-world applications. Examples of pictures in the dataset together with their class names are shown in figure 7.1.

7.2 In-house dataset

To train the segmentation model, custom training images were needed to accurately detect the instances of food. We took over 40 pictures with the compartmentalized plate containing a variety of foods that are present in the Food-101 dataset. These pictures were manually segmented, meaning a mask is drawn over the first, second and third instance of food with RGB values (1,1,1), (2,2,2) and (3,3,3) respectively. An example is shown in figure 7.2 with different color masks purely for visualization. The images together with their corresponding masks can now be fed into the Mask R-CNN network for training. When the images are segmented, the classification model can use the segmented parts and classify them, resulting in another classification dataset from the foods that we made ourselves.

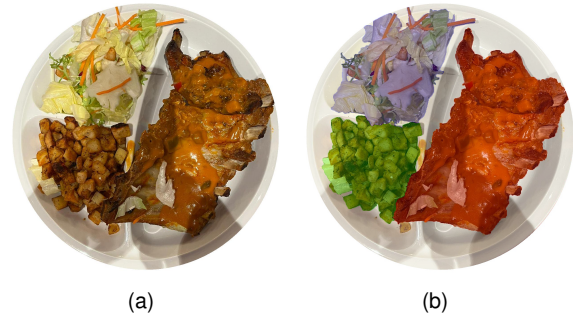


Figure 7.2: (a) A picture of ribs, potatoes and salad with (b) its three manually annotated masks

8 PIPELINE EXAMPLE

This chapter briefly discusses the whole pipeline, starting at the input image and finishing with the classified outputs.

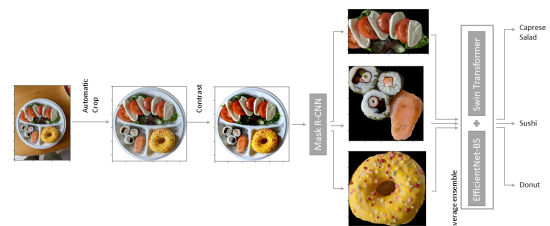


Figure 8.1: Example of an image going through the pipeline

The different steps of the process can be seen in figure 8.1. Firstly, a person takes a picture of the food that is divided on the compartmentalized plate. The image is then automatically cropped to the size of the compartmentalized plate by using Hough Circle Transformation to detect the circular shape of the plate (Bradski, 2000). Next, a contrast increase is applied to make edges and lines more distinguishable for the segmentation model. The three instances of food are then segmented and extracted from

the image, after which they can be classified. The three classes in this particular image are donut, caprese salad and sushi, which were all correctly predicted by the model as the highest probability.

9 RESULTS & DISCUSSION

We report results on the Food-101 dataset for classification and on the in-house dataset for both classification and segmentation.

	In-house Segmentation	
	mIoU	F1 score
Mask R-CNN	0.8800	0.9265

Table 9.1: Mean Intersection over Union and F1 score for the segmentation on the in-house dataset

In table 9.1 the segmentation accuracy with the Mask R-CNN on the in-house dataset is shown. It shows the mean Intersection over Union and F1 score (or dice coefficient) in percentage. Both evaluation metrics are described in chapter 5. These metrics show that the model able to adequately segment the food items, as they are both close to 1.

For classification, the top-1 and top-5 criteria are usually used as evaluation, where top-1 denotes the accuracy of the prediction with the highest probability being the correct class and top-5 denotes the correct class matching one of the five predictions with the highest probability, expressed in percentages.

	Food-101		
	top-1	top-5	Publication
FoodNet	72.12	91.61	Pandey et al.
WISeR	89.58	99.23	Martinel et al.
Swin Transformer	89.31	-	Liu et al.
			Fine-tuned by us
			Tan and Le
EffNet-B5	91.52	98.55	Fine-tuned by us
EffNet-B7	93.00	-	Tan and Le
Ensemble	93.52	99.13	Ours (proposed)
Grafit	93.70	-	Touvron et al.

Table 9.2: Top-1 and top-5 accuracy on the Food-101 dataset, - denotes not reported

Table 9.2, shows our results on the Food-101 dataset compared to prior papers. Note however that we omit works where extra (unavailable) data is used, for example the JFT-300M internal dataset from Google containing over 300 million images (Hinton et al., 2015). Our best results were produced by the average ensemble of the EfficientNet-B5 and Swin Transformer models. Vision transformers and convolutional neural networks approach image recognition very differently. While Vision transformers are not as translation equivariant as CNN's, their receptive fields are much bigger in lower layers due to using self-attention instead of convolutions. This is because self-attention is always computed over the whole image, while a convolutional kernel only attends to a small part of an image. This also results in higher representation similarity between layers that are far apart in vision transformers (Raghu et al., 2021). Intuitively we can conclude that ViT's and CNN's generate differing features when looking at the same image for classification. The average ensemble was able to combine the strengths of both architectures for a better classification accuracy ($\sim 2\%$ top-1 accuracy jump). This is further proven by looking at the different confusion matrices. The confusion matrix for the EfficientNet-B5, Swin Transformer and Average Ensemble model can be seen in figures B.1, C.1 and D.1. Tables 9.3, 9.4 and 9.5 show the five most confused categories. This shows that the EfficientNet-B5 and the Swin Transformer detect features differently, as the EfficientNet heavily struggles with distinguishing between lobster bisque/clam chowder and pork chop/steak, while the Swin Transformer can differentiate them more easily.

Most Confused: EfficientNet-B5		
Actual	Predicted	# of misscl- assifications
lobster bisque	clam chowder	39
pork chop	steak	39
bread pudding	apple pie	32
filet mignon	steak	31
chocolate mousse	chocolate cake	29

Table 9.3: Most confused categories by the EfficientNet-B5 model with actual and predicted categories and the number of wrong predictions

As can be seen in table 9.2, our approach is only slightly outperformed by (Touvron et al., 2020). It is worth mentioning that their 39 million parameter model was trained for 300 epochs, and while our proposed ensemble con-

Most Confused: Swin Transformer		
Actual	Predicted	# of misscl- assifications
steak	filet mignon	43
filet mignon	steak	26
steak	prime rib	24
apple pie	bread pudding	19
chocolate cake	chocolate mousse	19

Table 9.4: Most confused categories by the Swin Transformer model with actual and predicted categories and the number of wrong predictions

Most Confused: Average Ensemble		
Actual	Predicted	# of misscl- assifications
steak	filet mignon	31
filet mignon	steak	26
steak	prime rib	21
beef tartare	tuna tartare	19
chocolate mousse	chocolate cake	19

Table 9.5: Most confused categories by the Average Ensemble model with actual and predicted categories and the number of wrong predictions

sists of 228 Million parameters, it only had to be trained for a combined 24 epochs. Our approach is similar to (Pandey et al., 2017), where the authors fine-tuned three separate CNN's (GoogleNet, AlexNet and ResNet) and combined these three networks into an ensemble. In the future, improvements can be made to the average ensemble by using bigger versions of the EfficientNet architecture (B6, B7 and B8). However, our computing resources are limited and we were not able to fine-tune these bigger models in due time, as training time increases greatly for each bigger model (EfficientNet-B7 takes more than four times as long as EfficientNet-B5).

Table 9.6 shows the classification results on the in-house dataset. These are foods that we prepared ourselves that correspond to the classes that are present in the Food-101 dataset, as described in chapter 7. These accuracies show that our model trained on the Food-101 dataset translates well to the classification task of our own food on

In-house Classification			
	top-1	top-3	top-5
Ensemble	84.75	98.31	98.31

Table 9.6: Top-1, top-3 and top-5 classification accuracy on the in-house dataset

the compartmentalized plate.

10 CONCLUSION

The conclusion we can make from our results is three-fold. (i) The classification performance of the EfficientNet and Swin Transformer can be combined with an average ensemble, as it resulted in higher classification accuracy compared to using the models separately. (ii) The information that is learned from the Food-101 dataset translates well to custom pictures taken with a smartphone. (iii) The segmentation model provides good results and allows for very accurate segmenting of the different instances of food from the plate. We believe that we successfully reached the goal of this thesis, which was to automatically detect different types of food on a compartmentalized plate.

The main takeaway from our research and experiments is that the availability of good food-related datasets is scarce. Food-101 is one of the only (balanced) food datasets that contain a lot of pictures and can be used to train models for real-life applications. Still, this dataset only contains 101 categories of food and does not even contain half of the foods that an average person from Europe would consume daily. We believe more and bigger food-related datasets are essential for the progress of machine learning in the context of aiding dietary assessment.

ACKNOWLEDGEMENTS

Writing this thesis meant a lot of reading, coding and experimenting for both of us. We would like to thank the faculty of Group T for providing us with the ability to study and become engineers. Special thanks go to our co-promotor Sunilkumar Telagam Setti who helped us tremendously by meeting with us every week and giving us feedback as well as suggestions and useful ideas. Furthermore, we want to thank our friends and families for their continued support during this year of working on our thesis.

Manuscript under preparation: End-to-end approach to detect food on a compartmentalized plate

BIBLIOGRAPHY

- Bengio, Y. (2012). *Practical Recommendations for Gradient-Based Training of Deep Architectures*, pages 437–478. Springer Berlin Heidelberg, Berlin, Heidelberg.
- Bossard, L., Guillaumin, M., and Van Gool, L. (2014). Food-101 - Mining Discriminative Components with Random Forests. In Fleet, D., Pajdla, T., Schiele, B., and Tuytelaars, T., editors, *Computer Vision - ECCV 2014*, pages 446–461, Cham. Springer International Publishing.
- Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- Brock, A., De, S., Smith, S. L., and Simonyan, K. (2021). High-performance large-scale image recognition without normalization.
- Chang, J. and Jin, S. (2017). An efficient implementation of 2d convolution in cnn. *IEICE Electronics Express*, 14:20161134–20161134.
- Chaves, L. O., Domingos, A. L. G., Fernandes, D. L., Cerqueira, F. R., Siqueira-Batista, R., and Bressan, J. (2021a). Applicability of machine learning techniques in food intake assessment: A systematic review. *Critical Reviews in Food Science and Nutrition*, 0(0):1–18. PMID: 34323627.
- Chaves, L. O., Domingos, A. L. G., Fernandes, D. L., Cerqueira, F. R., Siqueira-Batista, R., and Bressan, J. (2021b). Applicability of machine learning techniques in food intake assessment: A systematic review. *Critical Reviews in Food Science and Nutrition*, 0(0):1–18. PMID: 34323627.
- Chen, L.-C., Papandreou, G., Kokkinos, I., Murphy, K., and Yuille, A. L. (2014). Semantic image segmentation with deep convolutional nets and fully connected crfs.
- Chen, L.-C., Zhu, Y., Papandreou, G., Schroff, F., and Adam, H. (2018). Encoder-decoder with atrous separable convolution for semantic image segmentation. In *ECCV*.
- Chollet, F. (2016). Xception: Deep learning with depthwise separable convolutions.
- Church, T. S., Thomas, D. M., Tudor-Locke, C., Katzmarzyk, P. T., Earnest, C. P., Rodarte, R. Q., Martin, C. K., Blair, S. N., and Bouchard, C. (2011). Trends over 5 Decades in U.S. Occupation-Related Physical Activity and Their Associations with Obesity. *PLOS ONE*, 6(5):1–7. Publisher: Public Library of Science.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houtsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale.
- Evans, C. (2005). Malnutrition in the elderly: a multifactorial failure to thrive. *Perm. J.*, 9(3):38–41.
- Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J., and Zisserman, A. (2010). The pascal visual object classes (voc) challenge. *International Journal of Computer Vision*, 88(2):303–338.
- Finestone, H. M., Greene-Finestone, L. S., Wilson, E. S., and Teasell, R. W. (1995). Malnutrition in stroke patients on the rehabilitation service and at follow-up: prevalence and predictors. *Arch. Phys. Med. Rehabil.*, 76(4):310–316.
- Foret, P., Kleiner, A., Mobahi, H., and Neyshabur, B. (2020). Sharpness-aware minimization for efficiently improving generalization.
- He, K., Gkioxari, G., Dollar, P., and Girshick, R. (2020). Mask r-cnn. *IEEE transactions on pattern analysis and machine intelligence*, 42(2):386–397.
- He, K., Zhang, X., Ren, S., and Sun, J. (2015). Deep residual learning for image recognition.
- Hinton, G., Vinyals, O., and Dean, J. (2015). Distilling the knowledge in a neural network.
- Joutou, T. and Yanai, K. (2009). A food image recognition system with multiple kernel learning. volume 285-288, pages 285 – 288.
- Kagaya, H., Aizawa, K., and Ogawa, M. (2014). Food detection and recognition using convolutional neural network. In *Proceedings of the 22nd ACM International Conference on Multimedia*, MM '14, pages 1085–1088, New York, NY, USA. Association for Computing Machinery.
- Kaiser, L., Gomez, A. N., and Chollet, F. (2017). Depthwise separable convolutions for neural machine translation.
- Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In Pereira, F., Burges, C., Bottou, L., and Weinberger, K., editors, *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc.

- Li, Y., Wei, C., and Ma, T. (2019). Towards explaining the regularization effect of initial large learning rate in training neural networks.
- Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C. L., and Dollár, P. (2014). Microsoft coco: Common objects in context.
- Liu, C., Cao, Y., Luo, Y., Chen, G., Vokkarane, V., and Ma, Y. (2016). Deepfood: Deep learning-based food image recognition for computer-aided dietary assessment.
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., and Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows.
- Long, J., Shelhamer, E., and Darrell, T. (2014). Fully convolutional networks for semantic segmentation.
- Martinel, N., Foresti, G. L., and Micheloni, C. (2016). Wide-slice residual networks for food recognition. *CoRR*, abs/1612.06543.
- Müller, O. and Krawinkel, M. (2005). Malnutrition and health in developing countries. *Cmaj*, 173(3):279–286.
- Nijhawan, R., Batra, A., Loyola-González, O., Kumar, M., and Jain, D. (2022). Food classification of indian cuisines using handcrafted features and vision transformer network. *SSRN Electronic Journal*.
- Pandey, P., Deepthi, A., Mandal, B., and Puhan, N. B. (2017). Foodnet: Recognizing foods using ensemble of deep networks. *IEEE Signal Processing Letters*, 24(12):1758–1762.
- Powell-Wiley, T. M., Poirier, P., Burke, L. E., Després, J.-P., Gordon-Larsen, P., Lavie, C. J., Lear, S. A., Ndumele, C. E., Neeland, I. J., Sanders, P., St-Onge, M.-P., and null null (2021). Obesity and cardiovascular disease: A scientific statement from the american heart association. *Circulation*, 143(21):e984–e1010.
- Raghu, M., Unterthiner, T., Kornblith, S., Zhang, C., and Dosovitskiy, A. (2021). Do vision transformers see like convolutional neural networks?
- Rashid, N., Dautta, M., Tseng, P., and Al Faruque, M. A. (2021). Hear: Fog-enabled energy-aware online human eating activity recognition. *IEEE Internet of Things Journal*, 8(2):860–868.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. (2018). Mobilenetv2: Inverted residuals and linear bottlenecks.
- Sarker, I. H. (2021). Machine learning: Algorithms, real-world applications and research directions. *SN Comput Sci*, 2(3):160.
- Simonyan, K. and Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition.
- Smith, L. N. (2015). Cyclical learning rates for training neural networks.
- Smith, L. N. and Topin, N. (2017). Super-convergence: Very fast training of neural networks using large learning rates.
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., and Rabinovich, A. (2014). Going deeper with convolutions.
- Tan, M. and Le, Q. V. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks.
- Tan, M. and Le, Q. V. (2021). Efficientnetv2: Smaller models and faster training. *CoRR*, abs/2104.00298.
- Touvron, H., Sablayrolles, A., Douze, M., Cord, M., and Jégou, H. (2020). Graft: Learning fine-grained image representations with coarse labels.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need.
- WHO (2020). The top 10 causes of death. <https://www.who.int/news-room/fact-sheets/detail/the-top-10-causes-of-death>.
- WHO (2021). Obesity and overweight. <https://www.who.int/en/news-room/fact-sheets/detail/obesity-and-overweight>.
- Willett, W. C., Manson, J. E., Stampfer, M. J., Colditz, G. A., Rosner, B., Speizer, F. E., and Hennekens, C. H. (1995). Weight, Weight Change, and Coronary Heart Disease in Women: Risk Within the 'Normal' Weight Range. *JAMA*, 273(6):461–465.
- Wortsman, M., Ilharco, G., Gadre, S. Y., Roelofs, R., Gontijo-Lopes, R., Morcos, A. S., Namkoong, H., Farhadi, A., Carmon, Y., Kornblith, S., and Schmidt, L. (2022). Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time.
- Wu, X., Fu, X., Liu, Y., Lim, E.-P., Hoi, S. C. H., and Sun, Q. (2021). A large-scale benchmark for food image segmentation.

- Xie, Q., Hovy, E. H., Luong, M., and Le, Q. V. (2019). Self-training with noisy student improves imagenet classification. *CoRR*, abs/1911.04252.
- Xu, J., Sun, X., Zhang, Z., Zhao, G., and Lin, J. (2019). *Understanding and Improving Layer Normalization*. Curran Associates Inc., Red Hook, NY, USA.
- Zhai, X., Kolesnikov, A., Houlsby, N., and Beyer, L. (2021). Scaling vision transformers.
- Zhang, X., Jia, F., Luo, S., Liu, G., and Hu, Q. (2014). A marker-based watershed method for x-ray image segmentation. *Computer Methods and Programs in Biomedicine*, 113(3):894–903.

A	Other classification experiments.	A-1
B	Confusion Matrix EfficientNet-B5	B-2
C	Confusion Matrix Swin Transformer	C-3
D	Confusion Matrix Average Ensemble	D-4

APPENDIX A : OTHER CLASSIFICATION EXPERIMENTS

	Food-101	
	top-1	Publication
EfficientNet-B7 noisy student	85.00	Xie et al. (2019) Fine-tuned by us
Neural network ensemble (EfficientNet-B5 + EfficientNetV2)	88.50	Ours
Treebagger (EfficientNet-B5 + EfficientNetV2)	89.00	Ours
EfficientNet-B5+slice convolution	90.30	Ours
EfficientNetV2	91.88	Tan and Le (2021) Fine-tuned by us
Treebagger (EfficientNet-(B5+B7 noisy student)+ Swin Transformer+EfficientNetV2)	93.11	Ours

Table A.1: Top-1 accuracy on Food-101 of other experiments not elaborated on in the paper

APPENDIX B : CONFUSION MATRIX EFFICIENTNET-B5

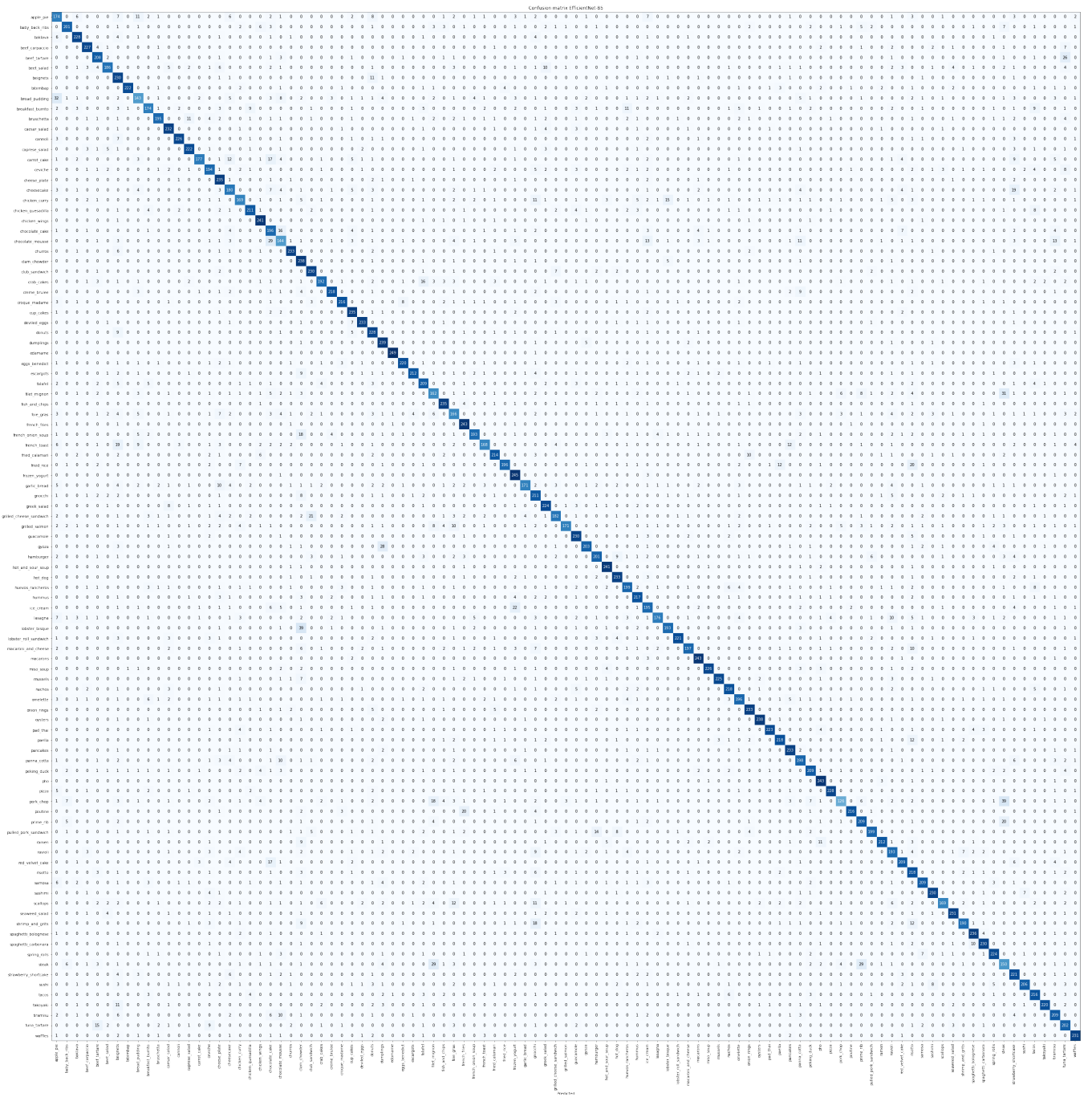


Figure B.1: Confusion Matrix EfficientNet-B5

APPENDIX C : CONFUSION MATRIX SWIN TRANSFORMER

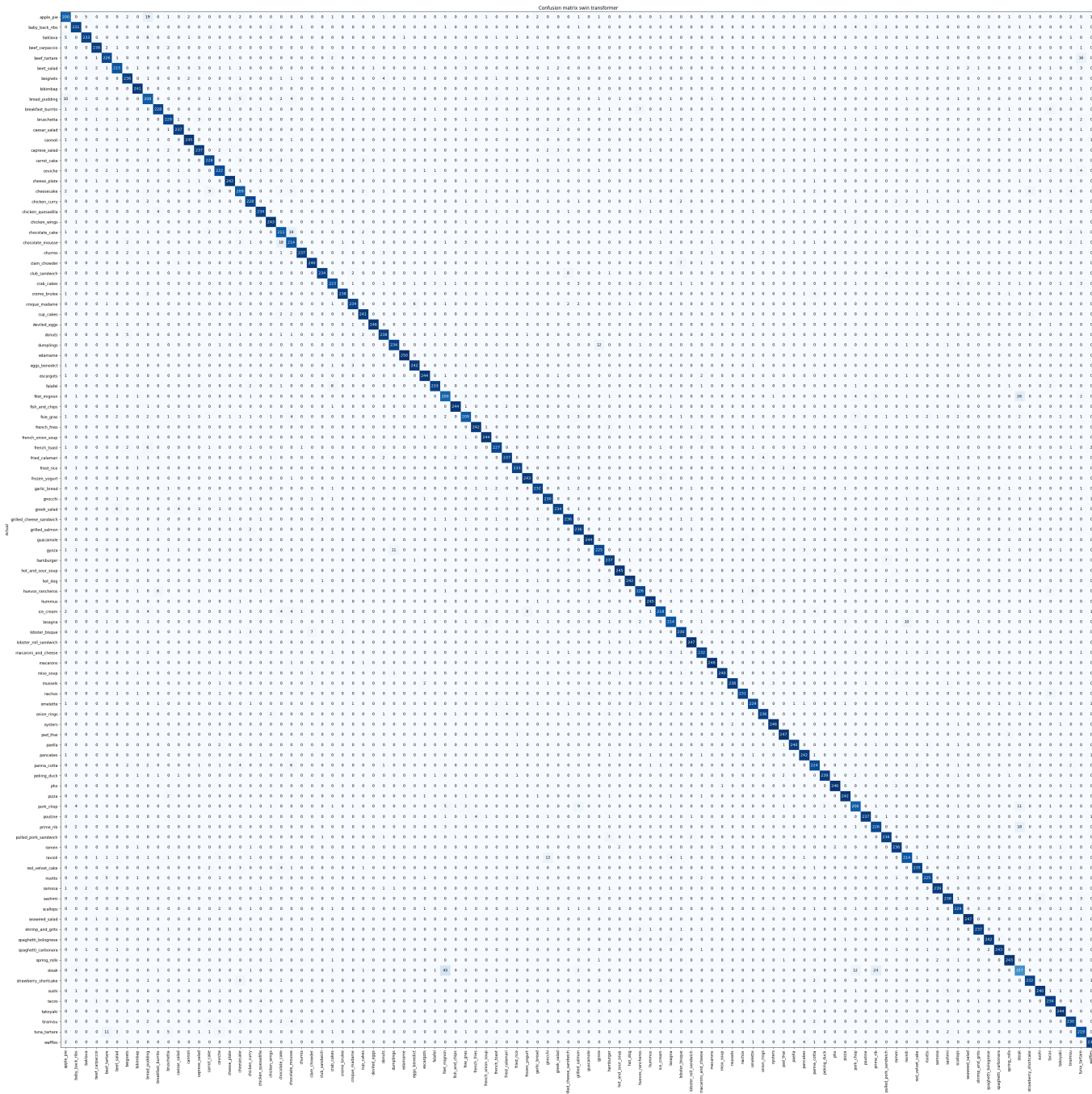


Figure C.1: Confusion Matrix Swin Transformer

APPENDIX D : CONFUSION MATRIX AVERAGE ENSEMBLE

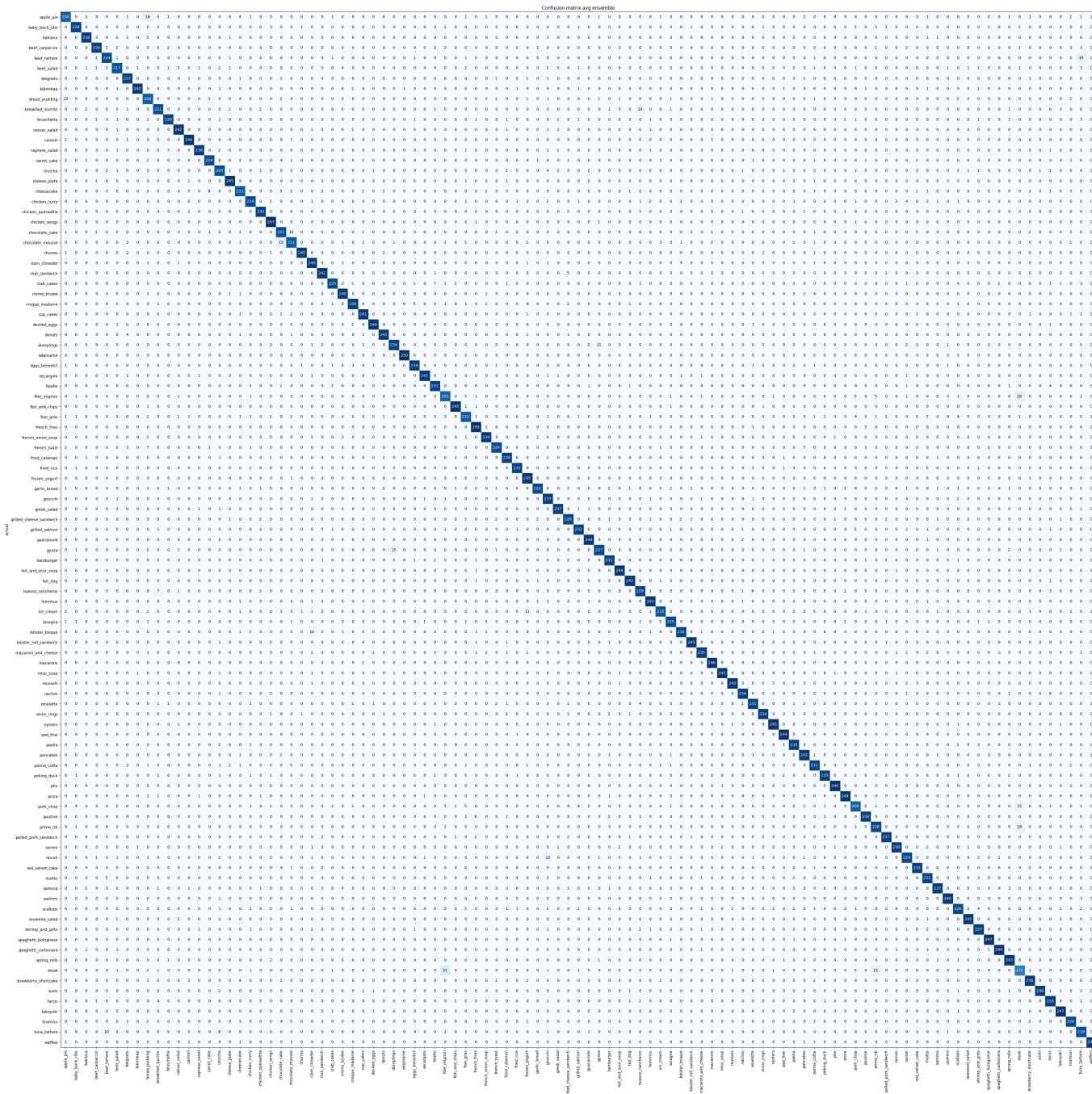


Figure D.1: Confusion Matrix Average Ensemble